

1.Introduction

Industrial robotic systems commonly use digital encoder servo motors to get accurate position feedback, torque control, velocity and safe gripping behavior. However, such encoder-based digital servos are relatively expensive and complex. This make unsuitable for most students and educational projects. Because almost learners afford and rely on low-cost hobby servos such as the MG90S and MG996R, but these do not have external feedback interfaces and advanced safety features. This limits students' ability to experiment with industrial robotics concepts using affordable hardware.

This project presents a **critical modification and control approach** that enables a standard hobby servo to similar with functions of a digital encoder servo by using its internal potentiometer feedback. Without adding any external sensors or components, the modified servo provides real-time position feedback to the microcontroller. So, the system can monitor actual joint motion rather than commanded angles. This approach significantly improves safety, reliability, and educational value with low cost and simplicity.

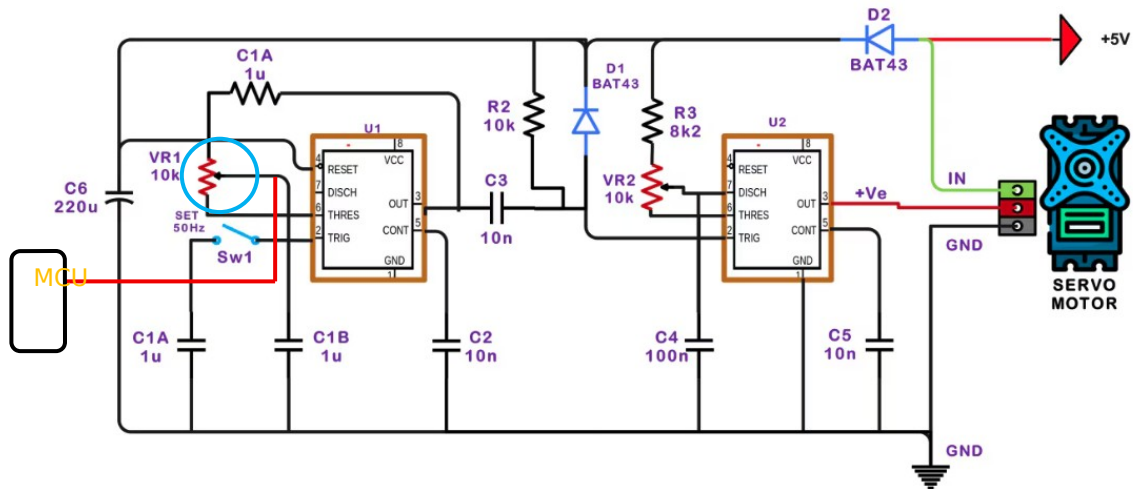
Using this feedback method, the servo can be guided manually in a teach mode and replayed in a play mode with recorded angle or position in tach mode, similar to industrial robotic arms. Additionally, stall detection based on feedback angle variation allows the gripper to stop or hold position automatically once an object is securely held that prevent the servo from driving to mechanical end stops. This behavior eliminates or stops a common cause of servo burnout in student projects, where grippers are typically driven to fixed angles such as 0° or 180° regardless of object size.

This proposed modification enables the application of higher-level robotic concepts, including forward kinematics (FK) and inverse kinematics (IK), by providing joint angle feedback. With reliable position data, future extensions of the system can be added as velocity and acceleration estimation, trajectory planning, and more advanced motion control techniques. These are essential for understanding real industrial robot behavior.

2.Potentiometer Feedback Extraction from Hobby Servo

In a hobby servo, an internal potentiometer is mechanically connected with the output shaft or end gear part and is used by the servo's internal control circuit to provide position feedback. The potentiometer consists of three terminals: two end terminals connected to a reference voltage and ground, and a middle wiper produces a continuously variable voltage output signal. In circuit diagram photo, the potentiometer is circled by blue color and wiper pin is middle arrow line that is used and connected as feedback line (red line) to microcontroller. In this work, the feedback signal is obtained by connection of the wiper terminal of the internal potentiometer and routing it to an analog

input pin of the microcontroller. This modification measures directly of the servo's actual angular position without any modifying the internal control circuitry or adding external sensors.



The wiper voltage relies and changes on with shaft rotation continuously, therefore making it, analog-to-digital conversion (ADC) is suitable by the microcontroller. With filtering and calibration (mapping), the system converts the analog potentiometer signal into a reliable joint angle. Since the potentiometer is already integrated in servo and mechanically aligned with the output shaft, this method provides accurate and low-cost position feedback interfaces. It is allowed to be comparable with encoder-based digital servos. Importantly, the original servo control loop remains intact, while the external microcontroller is taking real-time feedback for teach mode, playback, stall detection, and safety control.

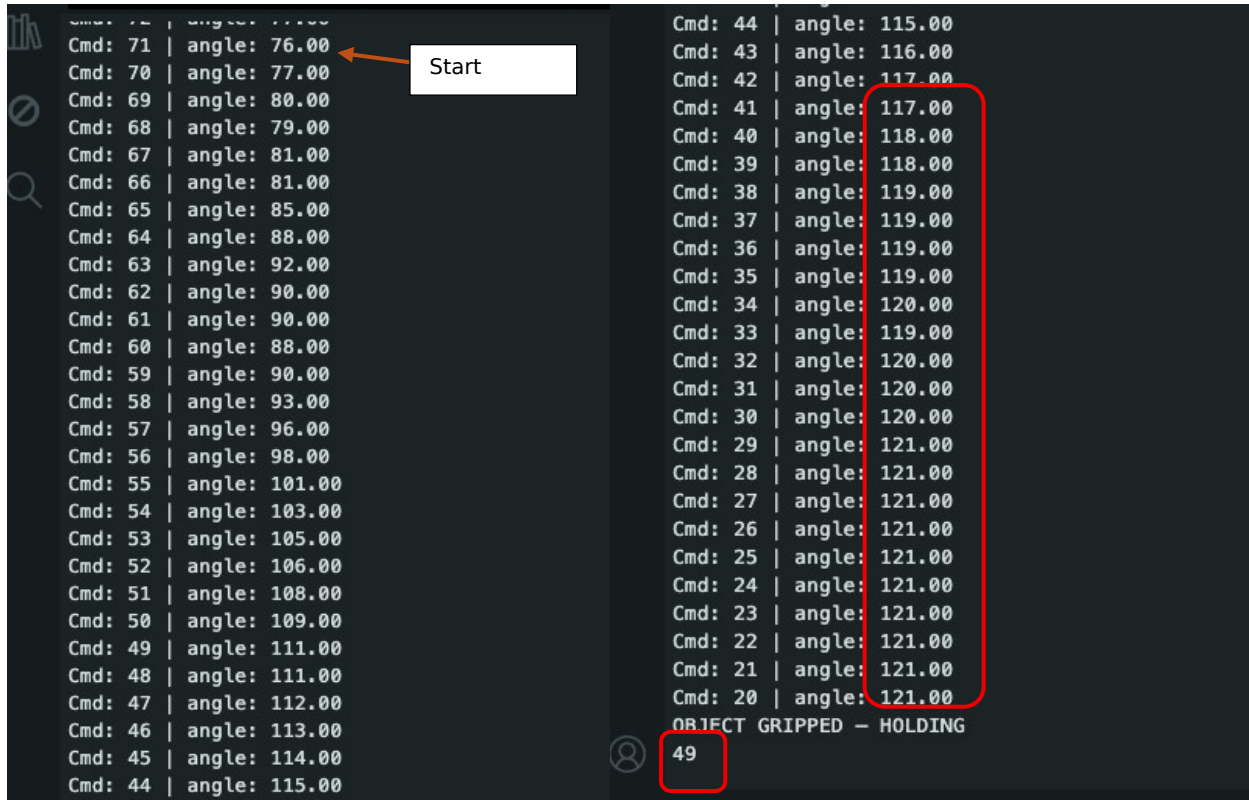
Potentiometer feedback and grounding

When the servo shaft is mechanically held or gripper gripped object before reaching end loop, the potentiometer value is not stable and has noise that is shown in figure. Because while holding the output shaft, error internally control circuit is larger than normal moving. Due to the command angle is still bigger than potentiometer angle.

$$\text{Error} = \text{Commanded position} - \text{Potentiometer voltage}$$

When the error is bigger, the controller drives the motor heavily that takes more currents. Motor current increases significantly that causes voltage drops along the shared ground path in circuit. Since the internal potentiometer uses same ground path and current flows through ground

paths. This causes variation in the ground reference between the servo and the MCU ADC. This effect on GND ground pin of potentiometer. As a result, the externally measured angle feedback is increased. It is still working with high latency but not perfect system. Because while holding at 71 degree and it stop at 20 cmd and holding angle is 49 degree. Moving each degree is taken 10ms. So, the latency is 500ms.

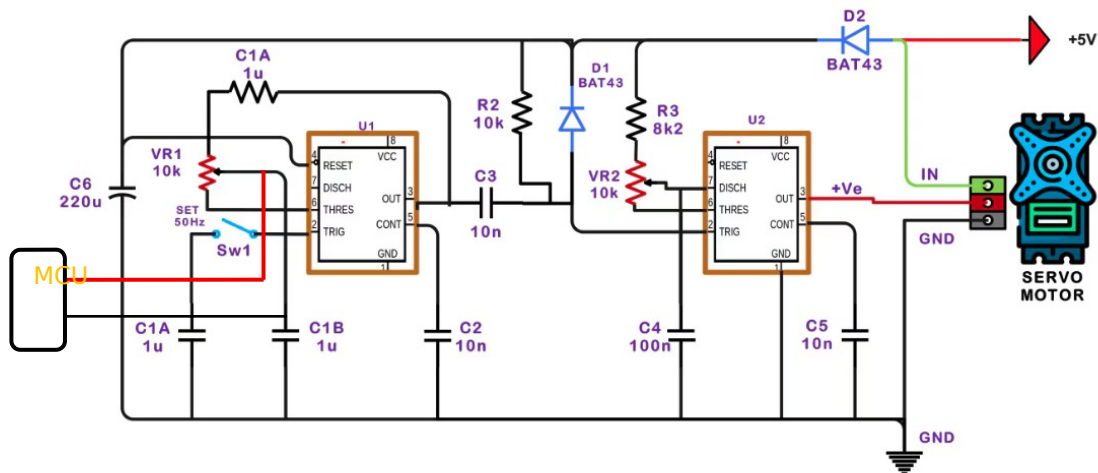
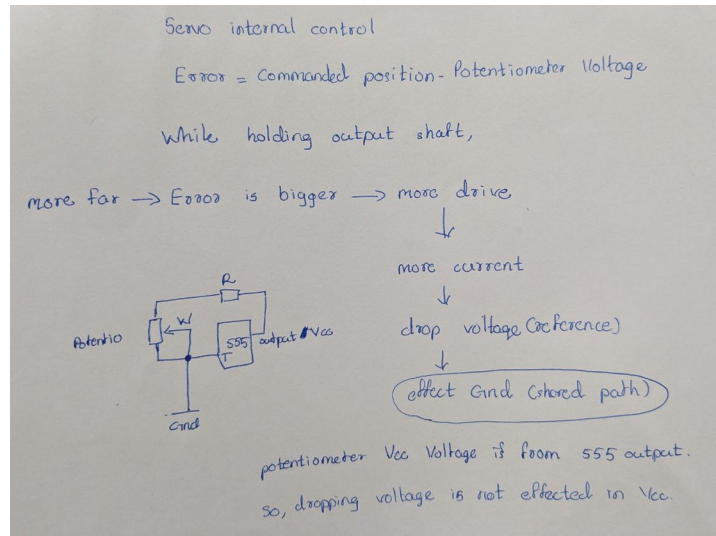


```
Cmd: 71 | angle: 76.00
Cmd: 70 | angle: 77.00
Cmd: 69 | angle: 80.00
Cmd: 68 | angle: 79.00
Cmd: 67 | angle: 81.00
Cmd: 66 | angle: 81.00
Cmd: 65 | angle: 85.00
Cmd: 64 | angle: 88.00
Cmd: 63 | angle: 92.00
Cmd: 62 | angle: 90.00
Cmd: 61 | angle: 90.00
Cmd: 60 | angle: 88.00
Cmd: 59 | angle: 90.00
Cmd: 58 | angle: 93.00
Cmd: 57 | angle: 96.00
Cmd: 56 | angle: 98.00
Cmd: 55 | angle: 101.00
Cmd: 54 | angle: 103.00
Cmd: 53 | angle: 105.00
Cmd: 52 | angle: 106.00
Cmd: 51 | angle: 108.00
Cmd: 50 | angle: 109.00
Cmd: 49 | angle: 111.00
Cmd: 48 | angle: 111.00
Cmd: 47 | angle: 112.00
Cmd: 46 | angle: 113.00
Cmd: 45 | angle: 114.00
Cmd: 44 | angle: 115.00

Cmd: 44 | angle: 115.00
Cmd: 43 | angle: 116.00
Cmd: 42 | angle: 117.00
Cmd: 41 | angle: 117.00
Cmd: 40 | angle: 118.00
Cmd: 39 | angle: 118.00
Cmd: 38 | angle: 119.00
Cmd: 37 | angle: 119.00
Cmd: 36 | angle: 119.00
Cmd: 35 | angle: 119.00
Cmd: 34 | angle: 120.00
Cmd: 33 | angle: 119.00
Cmd: 32 | angle: 120.00
Cmd: 31 | angle: 120.00
Cmd: 30 | angle: 120.00
Cmd: 29 | angle: 121.00
Cmd: 28 | angle: 121.00
Cmd: 27 | angle: 121.00
Cmd: 26 | angle: 121.00
Cmd: 25 | angle: 121.00
Cmd: 24 | angle: 121.00
Cmd: 23 | angle: 121.00
Cmd: 22 | angle: 121.00
Cmd: 21 | angle: 121.00
Cmd: 20 | angle: 121.00

OBJECT GRIPPED - HOLDING
49
```

In this system, the potentiometer is used based on the voltage divider principle, not as a direct resistance measurement device. A microcontroller cannot measure resistance directly; it can only measure voltage using its analog-to-digital converter (ADC). By connecting the two outer terminals of the potentiometer to a reference voltage (Vcc) and ground (GND), the wiper produces a voltage proportional to its mechanical position. This position-dependent voltage is read by the MCU and mapped to the corresponding angular position of the servo shaft.



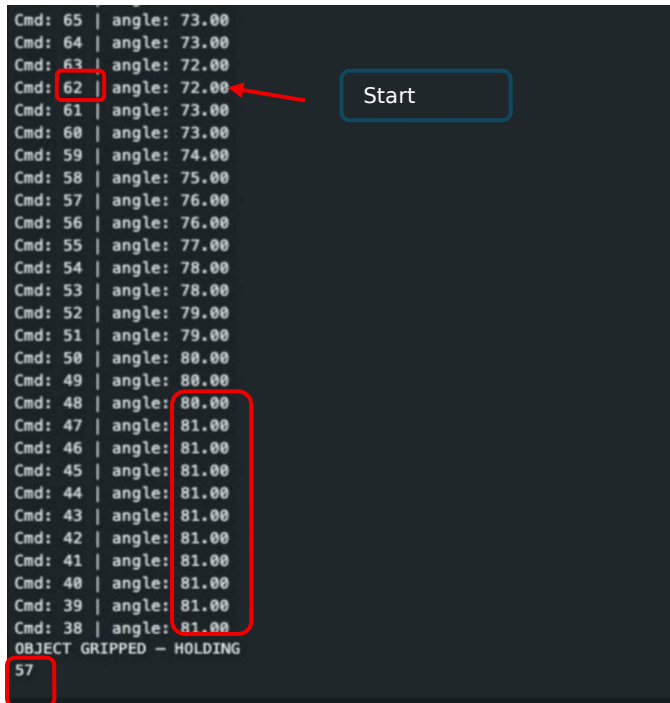
The **solution** is, the ground terminal of the potentiometer is directly connected to the MCU ground to ensure a common reference voltage between the sensor and the controller. Without a shared ground, the ADC readings become unstable and noisy because the measured voltage has no fixed reference. Proper grounding significantly improves measurement accuracy, response speed, and noise immunity, especially during motor load and stall conditions.

As the result, when I held the output shaft at 62 degree, stall detection is activated at 38. So, latency is only 200ms, and 100 ms is for counting stall (10 times). Then, it gripped at 57 degrees to get real hand torque.

```

Cmd: 65 | angle: 73.00
Cmd: 64 | angle: 73.00
Cmd: 63 | angle: 72.00
Cmd: 62 | angle: 72.00
Cmd: 61 | angle: 73.00
Cmd: 60 | angle: 73.00
Cmd: 59 | angle: 74.00
Cmd: 58 | angle: 75.00
Cmd: 57 | angle: 76.00
Cmd: 56 | angle: 76.00
Cmd: 55 | angle: 77.00
Cmd: 54 | angle: 78.00
Cmd: 53 | angle: 78.00
Cmd: 52 | angle: 79.00
Cmd: 51 | angle: 79.00
Cmd: 50 | angle: 80.00
Cmd: 49 | angle: 80.00
Cmd: 48 | angle: 80.00
Cmd: 47 | angle: 81.00
Cmd: 46 | angle: 81.00
Cmd: 45 | angle: 81.00
Cmd: 44 | angle: 81.00
Cmd: 43 | angle: 81.00
Cmd: 42 | angle: 81.00
Cmd: 41 | angle: 81.00
Cmd: 40 | angle: 81.00
Cmd: 39 | angle: 81.00
Cmd: 38 | angle: 81.00
OBJECT GRIPPED - HOLDING
57

```



RC Low-Pass Filtering and Signal Mapping

Why I used Low Pass Filter?

In the implemented servo system, a potentiometer is used to measure the angular position of the servo output shaft. However, the raw analog signal obtained from the potentiometer is affected by:

- Electrical noise
- ADC quantization noise
- Mechanical vibration of the servo

These effects may be **high-frequency variations** in the measured signal, which can cause:

- Jitter in angle estimation
- Unstable control behavior
- Incorrect stall detection in the gripping mechanism

To reduce these issues, a **low-pass filter** is applied to the sensor signal.

RC Low-Pass Filter Concept

An RC low-pass filter consists of a resistor (R) and a capacitor (C) arranged such that:

- Low-frequency signals pass through

- High-frequency noise is decreased

Continuous-time transfer function:

$$H(s) = \frac{1}{RCs + 1}$$

Where:

- R is resistance
- C is capacitance
- $\tau = RC$ is the time constant

This filter smooths rapid changes.

Digital implementation (software RC filter)

Instead of using a physical RC circuit, I implemented a **digital equivalent** of an RC low-pass filter using **exponential smoothing** because using software filter, reduce the complexity of hardware, can be use every student or leaner easily, no need to find and worry for hardware parts as resistor and capacitor.

Implemented equation:

$$y[k] = \alpha x[k] + (1 - \alpha) y[k-1]$$

Where:

- $x[k]$ = raw ADC reading
- $y[k]$ = filtered ADC value
- α = smoothing factor ($0 < \alpha < 1$)

This discrete-time filter behaves similarly to an RC low-pass filter.

```
filteredADC = alpha * rawADC + (1 - alpha) * filteredADC;
```

Relation between α and RC filter

In RC filter, large resistance and capacitance allow making heavy smoothing and small resistance and capacitance make weak filtering and fast response.

Large RC $\rightarrow$ heavy smoothing

Small RC $\rightarrow$ fast response

The smoothing factor α controls the filter response. This is equivalent to changing with the **RC constant**:

Small α (e.g. 0.05) $\rightarrow$ strong filtering, slower response

Large α (e.g. 0.5) $\rightarrow$ weaker filtering, faster response

In this system, $\alpha = 0.1$ was selected for noise reduction and acceptable response time for servo motion.

Signal Mapping (ADC $\rightarrow$ Angle)

The Arduino ADC produces a 10-bit digital value:

$$ADC \in [0, 1023]$$

However, the potentiometer in servo does not use the full ADC range due to mechanical limits and control circuit. All positional hobby servos are 180 degrees, but full range of potentiometer is 1032. Therefore, the usable range was found and used approximately:

$$ADC \in [60, 640]$$

To convert the filtered ADC value into a meaningful **angular position**, linear mapping is applied.

Mapping equation:

$$\theta = \frac{(ADC_{filtered} - ADC_{min})}{(ADC_{max} - ADC_{min})} \times 180^\circ$$

Code:

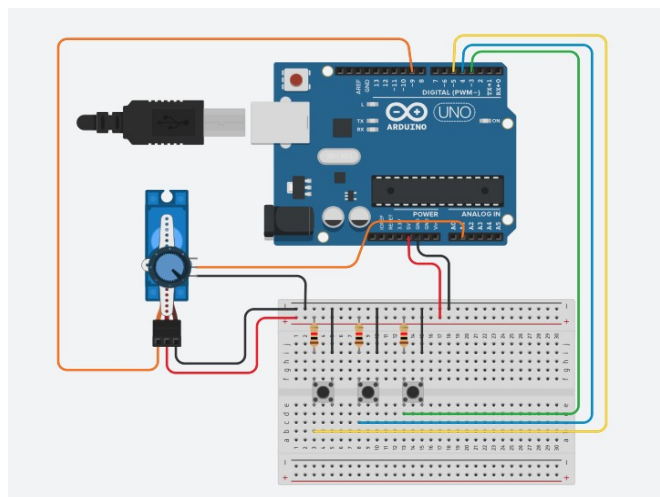
```
float angle = map(filteredADC, 60, 700, 0, 180);
```

This ensures estimation of correct physical angle and consistent feedback for analysis and stall detection.

Example Usages

Usage 1 (Teach-save-play concept mode)

The first application is Teach and play concept which is applied using this feedback data, that is included Memory Management & Mode Isolation in Teach-Play Servo Control. For the memory to save taught joint position or angles from teach mode, microcontroller ram is used with fixed-size array, it do **not** support dynamic memory management safely. It prevent buffer overflow on resource-constrained microcontrollers Later, we can upgrade and use EEP ROM of microcontroller or save in folder file if we use raspberry pi and OS version Controller.



Code:

```
// ----- TEACH STORAGE -----  
float taughtAngles[20];  
int taughtCount = 0;
```

This ensures; the amount of RAM (20 size of array) and execution timing remains deterministic. The system never allocates memory during runtime.

Mode isolation (Teach vs Play)

Why mode isolation is critical?

Teach mode and Play mode have **opposite objectives and functions**:

Mode	Motor	Human	Feedback
Teach	OFF	Controls joint	Read only
Play	ON	Passive	Monitor

Mode isolation between teach and play ensures that motor actuation and human-guided motion never occur simultaneously, preventing mechanical stress, invalid data collecting, and safety hazards. This state-design reflects real-world embedded robotic control principles.

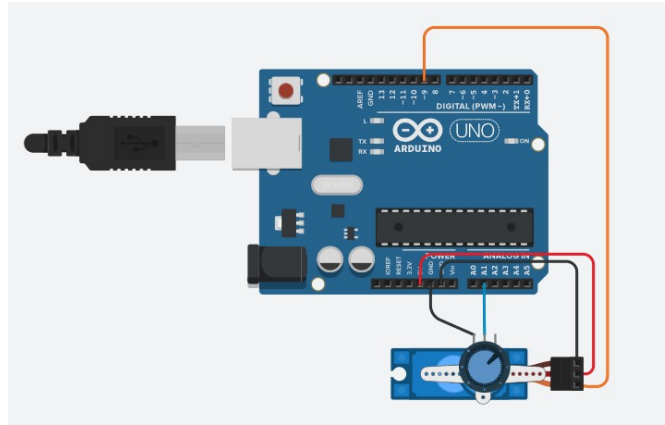
Usage 2 (auto stop gripping)

Stall Detection Using Angle Feedback and Stall Count

Normally, during gripping operation, the servo motor closes the gripper from an open position to the closed position fully because the size of the object being grasped is unknown. So, the servo continues to move after the object has been fully gripped. This makes:

- The motor is driven on excessive load
- Current increases
- Servo overheating and mechanical damage may occur
- Control circuit may be damaged

Therefore, a **stall detection mechanism** is required to automatically stop or hold the specific angle or position that the servo once the object is securely gripped. This makes saving servo and objective from over pressed damage. It works as real human hand gripping, the hand is touching or holding the objects.



Principle of stall detection

The stall detection algorithm is worked based on angle feedback from the servo potentiometer. While the servo is moving freely, the feedback angle changes continuously. **And then, when** the gripper contacts an object, the servo command continues, but the actual angle stops changing. This condition activates a **stall detection**. It works as capacitive sensor + encoder.

Angle difference calculation

IN each control loop iteration, the current feedback angle is compared with the previous feedback angle. Equation is

$$| \theta[k] - \theta[k-1] | < \varepsilon \text{ Where:}$$

- $\theta[k]$ = current feedback angle
- $\theta[k-1]$ = previous feedback angle
- ε = small threshold value

In this system:

```
const float ANGLE_EPS = 0.5;    // degrees
const int STALL_LIMIT = 10;    // number of loops
```

Angle_EPS is work as 0.5 constant for threshold value. This threshold prevents false stall detection caused by noise. It makes, instead of stopping the servo immediately when a small angle change is detected. The angle has not changed significantly for **10 consecutive loops**. The gripper is assumed to be holding an object surely.

The logic what I thought is:

- If the angle change is below the threshold → add to stallCount
- If movement resumes → reset stallCount (cause of noise or loose gripping)
- If stallCount reaches a predefined limit (eg 10) → stall confirmed

Code:

```
// ----- STALL DETECTION -----
if (abs(angle - lastAngle) < ANGLE_EPS) {
  stallCount++;
} else {
  stallCount = 0;
}
```

This ensures that short wrong detection does not trigger false stops caused by noises or loose gripping. Only **real non-motion** is counted as a stall. A stall is activated when the angles do not change for 10 consecutive cycles.

Servo shutdown or stop at specific angle(position) for safety

Once a stall is detected, the servo motor is immediately detach or stop the last angle to prevent damage. My recommendation is stopping the last angle for gripping and detach is for safety mode for wrong moving. This code is for detach while stall function is activated.

```
// ----- OBJECT GRIPPED -----
if (stallCount >= STALL_LIMIT) {
  Serial.println("OBJECT GRIPPED - STOP");
  myservo.detach(); // stop driving motor
  break;           // exit loop safely
}
```

This removes the PWM signal, cause dropping the motor torque and mechanical stress, and stopping heat generation.

Conclusion

In conclusion, this research focuses on learning and applying industrial robotics concepts to students with a low-cost, safe, and practical solution. By modifying cheap hobby servos into feedback-capable digital or analog hobby servos, the system reduces electronic waste by the burnt servos, improves

learning outcomes such as usage of feedback data and filter system, and learning the concepts between educational robotics and real-world industrial servo systems. This modification provides an effective teaching robotics fundamentals while being easy to implement.

Using this modification and implementation, learners can add and study control and analysis system. The RC low-pass filter represents a **first-order system**. It can be modeled using, DFT, Z-transform (discrete). Filtering improves signal quality before control or analysis. Mapping converts sensor signals into angles that can be what users wanted.